Emmalee Carpenter

10/25/2026

Written Report

Q1 Problem and scope

In A2, the tested baseline configuration missed a numeric Boolean injection because the permitted payloads changed query logic without producing the syntax errors used by its detection path.

- **Security property**: A user-supplied identifier should be treated as data and should not control the logical structure of the database query. Therefore, when user-supplied input is treated as SQL logic, SQLiFuzz should detect the injection attempt.
- **Baseline behavior**: SQLiFuzz does not detect a SQL injection even if a boolean expression is supplied within user-input and becomes SQL logic giving back information from the database.
- **Intended improvement**: Implement database-informed paired Boolean probes into SQLiFuzz's mechanics for supported numeric injection contexts.
- **Baseline paper**: I Putu Arya Dharmaadi, Van-Thuan Pham, Fadi Mohsen, and Fatih Turkmen. 2026. SQLiFuzz: Uncovering SQL Injection in Any Web Applications. Proc. ACM Softw. Eng. 3, FSE, Article FSE142 (July 2026). https://doi.org/10.1145/3808149
- **Authors' Artifact URL and Version**: Repository URL: https://github.com/websecfuzz/SQLiFuzz, SQLiFuzz commit: `1b7e2ede42af8e2b9d6d1c44358851e9d4616db9`
- **Reused A2 cases**: I reused a2_control.php and a2_filtered.php. These are webpages used to test different inputs and SQLiFuzz's capabilities. The control webpage allows for special characters while the filtered page rejects special characters but allows boolean expressions like AND and OR.

There was no change from the A2 proposal as once the fix was implemented, it worked cleanly. There were no problems and SQLiFuzz was able to recognize that method as a SQL injection.

Q2 How the method works

The implemented change extends SQLiFuzz's syntax-error detection with paired Boolean probes for numeric request parameters.

The method uses the request parameters, HTTP response bodies, and SQL queries recorded by the database proxy. It confirms a finding when the true and false probes produce consistently different responses and their payloads appear as unquoted SQL in comparable query structures.

Pseudocode:

```
For each eligible numeric URL or body parameter:
  Construct a true and false Boolean probe.
  Send each probe twice.
  Collect HTTP responses and corresponding database queries.

  If requests fail or responses vary between repetitions:
    Return inconclusive.

  If the probes are absent from SQL or appear only in literals/comments:
    Do not confirm injection.

  If query structures match and normalized responses differ:
    Report BOOLEAN_PAIR with supporting evidence.
```

The file `scripts/boolean_probe.py` (and `boolean_probe_OR.py`) is the main implementation. Within the file the `numeric_candidates()` function identifies plain integer parameters while excluding nested parameters and nonces. The `boolean_pair_probe()` function constructs and repeats the probes. `_replay()` copies and modifies the request, sends it through `send_HTTP_request()`, and collects database queries using `queries_in_window()`.

`payload_in_code()` and `mask_literals()` check whether the payload appears outside quoted strings, identifiers, and comments. `template()` compares query shapes after replacing literals and numbers. `normalize_body()` removes long hexadecimal tokens and normalizes whitespace. Finally, `judge()` checks the SQL evidence and response stability before returning confirmed, inconclusive, or unsupported.

For the primary case, the filter permits Boolean expressions while rejecting syntax-breaking characters. The prototype changes `id`=1 into:

```
1 AND 1=1
1 AND 1=0
```

The recorded queries contain these expressions as SQL logic. The corresponding responses contain `ROWS=1` and `ROWS=0`, respectively. Repeating the pair produces the same difference, so the prototype reports `BOOLEAN_PAIR`.

The method assumes that relevant database queries are observable, requests are associated with the correct logging window, the session remains valid, and responses are sufficiently stable. It will not help when an injection produces no observable response difference. The current implementation also does not generate the quote-breaking payloads needed for many string-context injections.

Q3 What you implemented

The main change is the Boolean probing helper and its integration into `test_SQLi()` in `src/SQLIFuzz.py`. After six ordinary mutations, the function attempts Boolean probing if it has

not already detected injection. A confirmed result is added to the existing result collection with its payload, SQL evidence, and response excerpts.

The configuration controls are:

- `SQLIFUZZ_BOOLEAN=0`: disable the added probing path.
- `SQLIFUZZ_BOOLEAN=1`: enable Boolean probing.
- `SQLIFUZZ_REQUIRE_STRUCTURE=1`: require SQL-context evidence.
- `SQLIFUZZ_SEED`: set the random seed.

The primary implementation is `scripts/boolean_probe.py`; the OR implementation is `src/boolean_probe_OR.py`. The runners install the appropriate helper under the runtime name `sqlifuzz/boolean_probe.py`. The variant runner restores AND afterward.

The file `patches/sqlifuzz-runtime.patch` identifies tracked changes relative to upstream. It also preserves inherited setup fixes, including the database-log directory correction, crawler compatibility change, container configuration mount, and launch-script shebangs. The reconstructed crawler helper is supplied separately in `src/crawler/utils.py`.

A nontrivial choice was to inspect raw SQL before masking quoted material. Searching only transformed SQL text could mistake a payload inside a string literal for executable SQL. The implementation therefore preserves raw query text and checks its context.

The upstream artifact supplies the original request handling, proxy infrastructure, mutation framework, and syntax-error detector. ChatGPT Work and Claude assisted with implementation, scripts, and documentation. My contributions included environment operation, trial execution, evidence collection, manual ground-truth checks, variant selection, and review. Attribution and verification are documented in notes/ai-use.md.

The central path runs now. Broader payload coverage, live safe-case testing, and scalability experiments remain unimplemented evaluation work.

Q4 What the pilot results show

The pilot contains 18 trials: three case groups, two configurations, and seeds 1–3. All trials have browser and proxy exit codes of zero.

Source inspection and A2 manual checks establish the primary vulnerability. For the OR variant, direct DVWA requests documented in `notes/ground_truth_boolean.md` independently returned `ROWS=5` and `ROWS=1`.

Case	Expected behavior	Baseline observed	Prototype observed	Mean elapsed seconds, baseline/prototype	Requests per trial, baseline/prototype
Primary	Baseline misses; prototype detects Boolean effect	0/3 detected	3/3, `boolean_pair`; `rows 1 versus 0`	201.08 / 184.05	70 / 10

Control	Both detect syntax-breaking injection	3/3, "syntax_error"	3/3, "syntax_error"	140.04 / 140.04	2/2
OR Variant	Baseline misses; prototype detects Boolean effect	0/3 detected	3/3, boolean_pair; rows 5 versus 1	207.57/140.03	70/10

All configurations used a 900-second outer timeout, seeds 1–3, and the same 70-slot mutation loop. The proxy used a 120-second inactivity shutdown when no fuzzing tasks remained. Boolean pairs were repeated twice, producing four replay requests per tested numeric parameter. Detection ends the search early.

Detection frequency counts completed trials reporting injection divided by completed trials for that configuration. Repeated probes and duplicate log messages are not separate detections. Regression reuses the control trials; the whole-path ablation reuses the primary comparison.

One traceable example is `raw/primary_case/prototype_run2/`. The corresponding command is:

- `scripts/run_trial.sh a2_filtered.php primary_case prototype 2 1 1`

The browser submits id=1. The final report, `final_result/FR-dvwa-manual-emmalee-VMware-Virtual-Platform-1792692875.txt`, records 1 AND 1=1, 1 AND 1=0, their executable SQL contexts, and `ROWS=1` versus `ROWS=0`. It reports ten requests, a request-duration sum of approximately 0.309509 seconds, and tool elapsed time of 140.035092 seconds. These values appear in the corresponding CSV row.

fuzzed_req counts completed calls through the shared request sender, including Boolean replays. The ten requests comprise six ordinary mutations and four Boolean requests. Browser login/navigation traffic is excluded. `processing_time_total_s` sums fetch durations; `total_run_time_s` includes the tool's waiting and idle shutdown.

These results establish that the new path executes and changes detection on the tested cases. They do not establish statistical significance or broad application coverage. A baseline miss means no finding within the tested search, rather than proof that detection is impossible.

Q5 Why the change matters

The primary case also serves as an ablation. When the boolean probing is disabled (key change), the baseline detected injection in 0/3 trials. When the probing is enabled, the prototype detected injection in 3/3 trials with recorded Boolean payloads and supporting SQL evidence. This supports the explanation that the new probing path addresses the tested syntax-error oracle limitation.

A separate test isolated the SQL structure requirement. The test function is defined in scripts/test_boolean_probe.py and runs `test_ablation_response_diff_only_gives_false_positive()`, output stored in `tests/results_of_tests.txt`. The output supplies stable, different responses without SQL evidence. With structure checking enabled, `judge()` returns inconclusive. Disabling that requirement changes the verdict to confirmed. This

demonstrates a false-positive risk of response-only classification at the decision-function level. It does not establish an application-level false-positive rate.

The matched control also serves as regression evidence. Both configurations detected the control in 3/3 trials through syntax errors, showing that even if the probe is disabled or enabled, past successful behavior is still maintained.

The implementation adds replay requests and database-log inspection. The primary prototype's elapsed times also vary substantially: one run took approximately 272 seconds, while the other two took approximately 140 seconds. Consequently, these results do not support a claim that the prototype is consistently faster.

Q6 A technical decision

A technical decision was to retain SQL-context evidence as a requirement for Boolean confirmation. A plausible alternative was to classify any stable true/false response difference as injection.

That alternative can fail when a safe application reflects the supplied input. The implementation addresses this explanation through literal/comment masking and the checks in `judge()`.

The relevant focused test evaluates:

```
judge(rounds, pt, pf, require_structure=True)
judge(rounds, pt, pf, require_structure=False)
```

For constructed responses without SQL evidence, the first returns inconclusive and the second returns confirmed. The saved output in `tests/results_of_tests.txt` records this test and the other 12 tests passing.

The October 25 retrospective review in `notes/decisions.md` documents the reasoning and evidence supporting retention of this check. It is explicitly retrospective and does not claim that the full explanation was recorded earlier.

The test addresses the reflected-input explanation within the decision function. A live safe reflected-input endpoint remains necessary to check request replay, query association, and classification together.

Q7 Final evaluation plan

The proposed final evaluation will compare the baseline, complete prototype, and response-only ablation. Six additional cases will be selected before further tuning: three vulnerable and three safe.

Question	Comparison and cases	Metric and experiment
Effectiveness	All three configurations on six additional cases	Detection frequency, missed known vulnerabilities, and false

		confirmations
Cost	Same cases and configurations	Verified request counts, elapsed time, and request-duration sum
Scalability	Baseline/prototype on targets with 1, 5, and 10 comparable endpoints	Elapsed time, requests, and fraction of known injectable endpoints detected
Remaining Failures	Quoted contexts, unstable responses, and safe reflected input	Findings, misses, inconclusive/unsupported verdicts, and execution failures

Expected outcomes will be established through source inspection and manual probes. Vulnerable cases will vary numeric-query structure and include a quoted context exposing the current payload limitation. Safe cases will use parameter binding, escaped quoted input, and strict integer validation.

The six-case comparison will use three seeds per configuration: 6 × 3 × 3 = 54 trials. Scaling will use three endpoint counts, two configurations, and three seeds: 18 trials. The proposed total is 72 trials.

Configurations will receive matching 900-second outer timeouts and documented search settings. The main ablation will keep probing enabled while disabling the SQL structure requirement. All failed, timed-out, inconclusive, and unsupported outcomes will remain visible. The result parser will be checked against raw exit codes so execution failures are not silently treated as completed negatives.

The existing primary, control, OR, and unit-test results are complete. Additional cases, live safe-case experiments, and scaling measurements remain planned.

Q8 Limits and remaining work

The main threat to generalization is that the pilot uses small, constructed DVWA cases with a single numeric parameter and explicit row-count output. The OR variant uses the same application logic with a different probe family. These results therefore cover a narrow input class.

The prototype may miss string-context injection because its current payload generation targets plain integer parameters. Dynamic page content may also make otherwise meaningful true/false differences unstable. Database-query association through timing windows may become less reliable under concurrent traffic.

The remaining work in order:

1. Verify reconstruction and the short test from a clean packaged checkout.
2. Add the held-out cases and independently establish their expected outcomes.
3. Test a live safe reflected-input case and the response-only ablation.
4. Run the additional effectiveness and scaling experiments.
5. Analyze unsuccessful outcomes, investigate timing variation, and revise the claims.

The 18 pilot trials total approximately 3,038 seconds, or 50.6 minutes: about 168.8 seconds per trial. At that average, 72 planned trials would take approximately 3.4 hours. Allowing 25% overhead gives approximately 4.2 hours. This estimate may understate the

scaling experiments; the 900-second timeout gives an upper bound of 18 hours of trial execution if every trial reaches the cap.

If resources become a blocker, the fallback is three additional cases—a numeric vulnerability, a safe reflected-input case, and a quoted-context limitation—tested across the three configurations and three seeds. Those 27 trials would still examine detection, false-positive behavior, and the structure check's contribution, but would not answer the scaling question.